

03-指令补充+样式绑定+计算属性+侦听器

一、指令补充

1. 指令修饰符

1.1 什么是指令修饰符？

所谓指令修饰符就是让指令的 功能更强大，书写更便捷

1.2 分类

1.2.1 按键修饰符

- @keydown.enter: 当enter键按下时触发
- @keyup.enter: 当enter键抬起时触发

代码演示:

```
1 <script setup>
2   // 处理按键函数
3   const onKeyDown = () => {
4     console.log('onKeyDown')
5   }
6 </script>
7 <input type="text" @keydown.enter="onKeyDown" />
```

1.2.2 事件修饰符

1. @事件名.stop —> 阻止冒泡
2. @事件名.prevent —> 阻止默认行为
3. @事件名.stop.prevent —> 可以连用 即阻止事件冒泡也阻止默认行为

```
1 <script setup>
2
3   // p标签的点击事件
4   const onPClick = () => {
5     console.log('onPClick')
6   }
7
8   // div标签的点击事件
```

```

9     const onDivClick = () => {
10         console.log('onDivClick')
11     }
12 </script>
13
14 <template>
15     <!-- 事件修饰符 -->
16     <div @click="onDivClick">
17         <!-- .prevent: 阻止默认行为 -->
18         <a href="https://baidu.com" @click.prevent>百度一下</a>
19         <!-- .stop: 阻止冒泡, 同名事件不会向上级传递 -->
20         <p @click.stop="onPClick"></p>
21
22         <!-- 修饰符的链式调用: 表明两个同时阻止 -->
23         <a href="https://baidu.com" @click.stop.prevent>百度一下</a>
24     </div>
25 </template>
26
27 <style>
28     div {
29         width: 400px;
30         height: 200px;
31         background: plum;
32     }
33     div a {
34         display: block;
35         width: 100px;
36         text-decoration: none;
37         background: tomato;
38         text-align: center;
39         color: #fff;
40     }
41     div p {
42         width: 200px;
43         height: 100px;
44         background: rebeccapurple;
45     }
46 </style>

```

1.2.3 v-model修饰符

- v-model.trim —> 去除首尾空格
- v-model.number —> 尝试用parseFloat()转数字
- v-model.lazy —> 失去焦点时同步数据, 而不是输入时同步数据

```

1  <script setup>
2    import { reactive } from 'vue'
3    const goods = reactive({
4      name: '',
5      price: ''
6    })
7  </script>
8
9  <template>
10   <div>
11     <!-- .lazy修饰符 -->
12     <!-- 名称:
13     <input
14       type="text"
15       v-model.lazy="goods.name" /><br /><br /> -->
16
17     <!-- .trim修饰符 -->
18     名称:
19     <input
20       type="text"
21       v-model.trim="goods.name" /><br /><br />
22
23     <!-- .number修饰符 -->
24     价格:
25     <input
26       type="text"
27       v-model.number="goods.price" /><br /><br />
28   </div>
29 </template>
30
31 <style scoped></style>

```

1.3 总结

1. 如何监听回车键?

答: 1、@keydown.enter -> 回车按下

2、@keyup.enter -> 回车抬起

2. 如何阻止默认行为、阻止冒泡?

答: 1、@click.prevent -> 阻止默认行为

2、@click.stop -> 阻止冒泡

3、@click.prevent.stop -> 可以链式调用, 二者都阻止

3. v-model的3个修饰符的作用是什么?

答: 1、.lazy -> 失去焦点再同步

2、.trim -> 去除首尾空格

3、.number -> 尝试转数字

2. v-model用在其他表单元素上

2.1 讲解内容

常见的表单元素都可以用 v-model 绑定关联, 作用是可以快速 获取 或 设置 表单元素的值

它会根据 控件类型 自动选取 正确的属性 来更新元素

- | | | | |
|---|------|-----------------------------|---------------------------------|
| 1 | 输入框 | <code>input:text</code> | —> <code>value</code> |
| 2 | 文本域 | <code>textarea</code> | —> <code>value</code> |
| 3 | 下拉菜单 | <code>select</code> | —> <code>value</code> |
| 4 | 单选框 | <code>input:radio</code> | —> <code>value</code> |
| 5 | 复选框 | <code>input:checkbox</code> | —> <code>checked / value</code> |

2.2 代码准备

```
1  <script setup>
2    import { ref } from 'vue'
3    // 自我介绍
4    const intro = ref('')
5
6    // 收集城市
7    const city = ref('SH')
8
9    // 收集血型
10   const blood = ref('ab')
11
12   // 是否同意用户协议
13   const isAgree = ref(false)
14
15   // 收集爱好
16   const hobby = ref(['ZQ', 'PB'])
17 </script>
18
19 <template>
20   <div>
21     <!-- 文本域 -->
22     <textarea
23       v-model="intro"
24       cols="30"
```

```
25     rows="4"
26     placeholder="请输入自我介绍"></textarea>
27
28 <br />
29 <br />
30 <!-- 下菜单 -->
31 <select v-model="city">
32     <option value="BJ">北京</option>
33     <option value="SH">上海</option>
34     <option value="SZ">深圳</option>
35     <option value="HZ">杭州</option>
36 </select>
37 <br />
38 <br />
39 <!-- 单选框：多个当中只能选择一个，需要给单选框手动添加 value 属性 -->
40 <input
41     type="radio"
42     value="a"
43     v-model="blood" />A
44 <input
45     type="radio"
46     value="b"
47     v-model="blood" />B
48 <input
49     type="radio"
50     value="ab"
51     v-model="blood" />AB
52 <input
53     type="radio"
54     value="o"
55     v-model="blood" />O
56
57 <br />
58 <br />
59 <input
60     type="checkbox"
61     v-model="isAgree" />是否同意用户协议
62
63 <br />
64 <br />
65 <input
66     v-model="hobby"
67     type="checkbox"
68     value="LQ" />篮球
69 <input
70     v-model="hobby"
71     type="checkbox"
```

```
72     value="ZQ" />足球
73     <input
74       v-model="hobby"
75       type="checkbox"
76       value="YMQ" />羽毛球
77     <input
78       v-model="hobby"
79       type="checkbox"
80       value="PPQ" />乒乓球
81     <br />
82     <input
83       v-model="hobby"
84       type="checkbox"
85       value="PB" />跑步
86     <input
87       v-model="hobby"
88       type="checkbox"
89       value="YY" />游戏
90     <input
91       v-model="hobby"
92       type="checkbox"
93       value="PLT" />普拉提
94     <input
95       v-model="hobby"
96       type="checkbox"
97       value="LDW" />拉丁舞
98   </div>
99 </template>
100
101 <style scoped></style>
```

2.3 总结

1. v-model如何收集下拉列表的值？

答: v-model写在select上, 关联是选中option的`value`

2. v-model如何收集单选框的值？

答: 给单选框添加value属性, v-model收集选中单选框的value

3. v-model作用在复选框上组要注意什么？

答: 1、一个复选框, v-model绑定 布尔值, 关联 checked 属性

2、一组复选框, v-model绑定 数组, 关联 value 属性,

给复选框 手动添加 value

二、样式绑定

1. 基本介绍

1.1 概述

为了方便开发者进行样式控制,Vue 扩展了 v-bind 的语法,可以针对 **class 类名** 和 **style 行内样式** 两个属性进行控制,进而通过数据控制元素的样式

1.2 分类

1.2.1 操作class

1.2.2 操作style

2. 操作class

2.1 语法

```
:class = "三元表达式 / 对象"
```

2.2 三元表达式

```
1 <p :class="条件 ? '类名1' : '类名2'"></p>
```

2.3 对象语法

当class动态绑定的是**对象**时, **键就是类名, 值就是布尔值**, 如果值是**true**, 就添加这个类, 否则删除这个类

```
1 <p class="box" :class="{ 类名1: 布尔值1, 类名2: 布尔值2 }"></p>
```

适用场景: 一个类名, 来回切换

2.4 静态class与动态class共存

静态class与动态class共存可以**共存**, 二者会**合并**

```
1 <p class="item" :class="条件 ? '类名1' : '类名2'"></p>
```

2.5 代码演示

```
1  <script setup>
2    import { ref } from 'vue'
3
4    // 是否处于激活
5    const isActive = ref(true)
6  </script>
7
8  <template>
9    <div>
10     <!-- 1. 三元绑定 -->
11     <p :class="isActive ? 'active' : ''>Active1</p>
12     <!-- 2. 对象绑定 -->
13     <p :class="{ active: isActive }">Active2</p>
14     <!-- 3. 静态class与动态class共存 -->
15     <p class="item" :class="{ active: isActive }">Active3</p>
16   </div>
17 </template>
18
19 <style>
20   .active {
21     color: red;
22   }
23 </style>
```

2.6 总结

1. 如何通过class动态控制元素的样式?

答: 1、 `:class="三元表达式"`

2、 `:class="{ 类名: 布尔值 }"`, 布尔值为true则添加类名;
否则移除

2. 静态和动态class可以共存吗?

答: 可以 共存, 二者会合并

3. 比特就业课导航切换

3.1 效果

比特就业课官网顶部导航栏切换

<https://bitejiuyeye.com/index>

3.2 静态代码(可复制)

代码块

```
1  <script setup>
2    // 导航列表
3    const navList = [
4      {
5        id: 101001,
6        name: '首页'
7      },
8      {
9        id: 101002,
10       name: '鹏哥C语言'
11     },
12     {
13       id: 101003,
14       name: '比特就业课'
15     },
16     {
17       id: 101004,
18       name: '有问必答'
19     },
20     {
21       id: 101005,
22       name: '关于比特'
23     }
24   ]
25 </script>
26
27 <template>
28   <div class="bit-header">
29     <div class="main">
30       <div class="logo">
31         <a
32           href="https://bitejiuyeke.com/index"
33           target="_blank">
34           
39         </a>
```

```
40     </div>
41     <ul>
42         <li>
43             <a
44                 href="#"
45                 class="active"
46                 >首页</a
47             >
48         </li>
49     </ul>
50 </div>
51 </div>
52 </template>
53
54 <style lang="scss">
55     * {
56         margin: 0;
57         padding: 0;
58     }
59     a {
60         display: block;
61         padding: 12px 16px;
62         margin: 0 15px;
63         text-decoration: none;
64         color: #909399;
65         transition: color 1s;
66         font-family: Microsoft YaHei UI;
67         &:hover {
68             color: #303133;
69         }
70     }
71     .bit-header {
72         background: #fff;
73         box-shadow: 0 3px 6px 0 rgba(7, 35, 68, 0.15);
74         .main {
75             display: flex;
76             align-items: center;
77             width: 1200px;
78             height: 80px;
79             margin: 0 auto;
80             .logo {
81                 margin-right: 37px;
82             }
83             ul {
84                 display: flex;
85                 align-items: center;
86                 list-style: none;
```

3.3 完整代码

```
1  <script setup>  
2    import { ref } from 'vue'  
3  
4    const currentIndex = ref(0)  
5  </script>  
6  
7  <template>  
8    <div class="bit-header">  
9      <div class="main">  
10       <div class="logo"></div>  
11       <ul>  
12         <li  
13           v-for="(item, index) in navList"  
14           :key="item.id"  
15           @click="currentIndex = index">  
16             <a  
17               href="#"  
18               :class="{ active: index === currentIndex }"  
19               >{{ item.name }}</a>  
20           >  
21         </li>  
22       </ul>  
23     </div>  
24   </div>  
25 </template>
```

4. 操作style

4.1 语法

```
1  <div class="box" :style="{ CSS属性名1: CSS属性值, CSS属性名2: CSS属性值 }"></div>  
2  
3  <div class="box" :style="样式对象"></div>
```

4.2 代码演示

代码块

```
1
```

```

2  <script setup>
3    import { reactive, ref } from 'vue'
4
5    const colorStr = ref('red')
6    const bgColor = ref('pink')
7
8    const styleObj = reactive({
9      color: 'blue',
10     background: 'yellowgreen'
11   })
12 </script>
13
14 <template>
15   <p :style="{ color: colorStr, background: bgColor }">小比特</p>
16
17   <p :style="styleObj">大梦想</p>
18 </template>

```

4.3 可折叠面板(可复制)

代码块

```

1  <script setup>
2    import { ref } from 'vue'
3
4    const visible = ref(true)
5  </script>
6
7  <template>
8    <h3>可折叠面板</h3>
9    <div class="panel">
10      <div class="title">
11        <h4>自由与爱情</h4>
12        <span
13          class="btn"
14          @click="visible = !visible">
15          {{ visible ? '收起' : '展开' }}
16        </span>
17      </div>
18      <div
19        class="container"
20        v-show="visible">
21        <p>生命诚可贵,</p>
22        <p>爱情价更高。</p>
23        <p>若为自由故,</p>
24        <p>两者皆可抛。</p>

```

```
25     </div>
26 </div>
27 </template>
28
29 <style lang="scss">
30   body {
31     background: #ccc;
32   }
33
34   #app {
35     width: 400px;
36     margin: 20px auto;
37     padding: 1em 2em 2em;
38     box-shadow: 3px 3px 3px rgba(0, 0, 0, 0.5);
39     background: #fff;
40   }
41   #app h3 {
42     text-align: center;
43   }
44
45   .panel {
46     .title {
47       display: flex;
48       justify-content: space-between;
49       align-items: center;
50       padding: 0 1em;
51       border: 1px solid #ccc;
52     }
53     .title h4 {
54       line-height: 2;
55       margin: 0;
56     }
57     .container {
58       border: 1px solid #ccc;
59       padding: 0 1em;
60     }
61     .btn {
62       cursor: pointer;
63     }
64   }
65 </style>
```

4.4 完整代码

```
1 <div
```

```
2     class="title"
3     :style="{ borderBottomColor: visible ? 'transparent' : '#ccc' }"></div>
```

4.5 总结

1. 如何给元素动态绑定style?

答: 1、`:style="{ 属性名1: 表达式1, 属性名2: 表达式2, ... }"`

2、`:style="样式对象"`

三、计算属性

1. 基本使用

1.1 概念

基于现有的数据, 计算出来的新数据; 当现有的数据变化, 会自动重新计算

1.2 语法

1. 使用 **computed** 函数, 计算得到一个新数据进行展示

```
1  const 新数据 = computed(() => {
2    // some code ...
3    return 结果
4  })
5
6  // 商品列表(原始数据)
7  const goodsList = ref([
8    { id: 1, name: '篮球', num: 1 },
9    { id: 2, name: '玩具', num: 3 },
10   { id: 3, name: '书籍', num: 2 }
11  ])
```

1.3 案例

比如我们可以使用计算属性实现下面这个业务场景

```
// 商品列表(原始数据)
const goodsList = ref([
  { id: 1, name: '篮球', num: 1 },
  { id: 2, name: '玩具', num: 3 },
  { id: 3, name: '书籍', num: 2 }
])
```

+

```
// 计算总数
const totalNum = computed(() => {
  // 编写计算逻辑
  return 结果
})
```

→

比特的礼物清单

名字	数量
篮球	1个
玩具	3个
书籍	2个

礼物总数: ? 个 (计算属性名)

展示计算结果

1.4 静态代码(可复制)

```

1  <script setup>
2
3  import { ref } from 'vue'
4
5  // 商品列表(原始数据)
6  const goodsList = ref([
7    { id: 1, name: '篮球', num: 1 },
8    { id: 2, name: '玩具', num: 3 },
9    { id: 3, name: '书籍', num: 2 }
10  ])
11
12 </script>
13 <template>
14   <h3>比特人的礼物清单</h3>
15   <table>
16     <tr>
17       <th>名字</th>
18       <th>数量</th>
19     </tr>
20     <tr>
21       v-for="item in goodsList"
22       :key="item.id">
23       <td>{{ item.name }}</td>
24       <td>{{ item.num }}个</td>
25     </tr>
26   </table>
27   <!-- 目标: 统计求和, 展示礼物总数 -->
28   <p>礼物总数: ? 个</p>
29 </template>
30
31 <style>
32   table {
33     width: 350px;
34     border: 1px solid #333;
35   }
36   table th,
37   table td {
38     border: 1px solid #333;
39   }
40   table td {
41     text-align: center;
42   }
43 </style>

```

1.5 完整代码

```

1  <script setup>
2    import { computed, ref } from 'vue'
3    // 商品列表(原始数据)
4    const goodsList = ref([
5      { id: 1, name: '篮球', num: 1 },
6      { id: 2, name: '玩具', num: 3 },
7      { id: 3, name: '书籍', num: 2 }
8    ])
9
10   // 由于这里只有原始数据, 没有直接给我们提供商品总数量
11   // 但是我们可以基于现有的 goodsList 计算得到总数量
12   // 那么推荐把总数量声明为 计算属性
13   const totalNum = computed(() => {
14     return goodsList.value.reduce((prev, item) => prev + item.num, 0)
15   })
16 </script>
17
18 <template>
19   <h3>比特人的礼物清单</h3>
20   <table>
21     <thead>
22       <tr>
23         <th>名字</th>
24         <th>数量</th>
25       </tr>
26     </thead>
27     <tbody>
28       <tr
29         v-for="item in goodsList"
30         :key="item.id">
31         <td>{{ item.name }}</td>
32         <td>{{ item.num }}</td>
33       </tr>
34     </tbody>
35   </table>
36   <p>礼物总数: {{ totalNum }} 个</p>
37 </template>
38
39 <style>
40   table {
41     width: 350px;
42     border: 1px solid #333;
43   }
44   table th,
45   table td {
46     border: 1px solid #333;
47   }

```



```
48     table td {
49         text-align: center;
50     }
51 </style>
```

1.6 注意

1. 计算属性必须有返回值
2. 使用和 ref/reactive 数据一样, 可用于 插值, 也可 配合指令

1.7 总结

1. 何时用计算属性?

答: 基于现在数据计算得到新数据的时候

2. 语法?

```
1  const 新数据 = computed(() => {
2      return 计算结果
3  })
```

2. 计算属性 VS 普通函数

2.1 计算属性

2.1.1 作用

封装了一段对于基于现有数据, 计算求得一个**新数据**

2.1.2 语法

1. 写在computed函数中, 必须返回
2. 作为属性, 直接使用
 - js中获取计算属性: 计算属性.value
 - 模板中使用计算属性: {{ 计算属性 }} 或 配合指令

2.2 普通函数

2.2.1 作用

封装一段js代码, 用于调用以**处理业务逻辑**。

2.2.2 语法

1. 直接在script下定义

2. 作为函数调用

- js中调用：函数名(实参列表)
- 模板中调用 {{ 函数名 (实参列表) }} 或者 @事件名= “方法名(实参列表)”

3. 计算属性的优势

- 缓存特性(提升性能)，函数没有缓存特性
- 计算属性会对计算出来的结果缓存，再次使用直接读取缓存
- 只有依赖项变了，才会自动重新计算，并再次缓存新数据

```
1  <script setup>
2    import { ref, computed } from 'vue'
3
4    // 商品列表(原始数据)
5    const goodsList = ref([
6      { id: 1, name: '篮球', num: 1 },
7      { id: 2, name: '玩具', num: 3 },
8      { id: 3, name: '书籍', num: 2 }
9    ])
10
11    // 计算总数
12    const totalNum = computed(() => {
13      return goodsList.value.reduce((prev, item) => prev + item.num, 0)
14    })
15  </script>
16  <template>
17    <div id="app">
18      <h3>比特人的礼物清单</h3>
19      <table>
20        <tr>
21          <th>名字</th>
22          <th>数量</th>
23        </tr>
24        <tr
25          v-for="item in goodsList"
26          :key="item.id">
27          <td>{{ item.name }}</td>
28          <td>{{ item.num }}个</td>
29        </tr>
30      </table>
31
32      <!-- 目标：统计求和，求得礼物总数 -->
33      <p>礼物总数：{{ totalNum }} 个</p>
34    </div>
35  </template>
```

```

36
37 <style>
38   #app {
39     width: 240px;
40     margin: 100px auto;
41   }
42   table {
43     border: 1px solid #000;
44     text-align: center;
45     width: 240px;
46   }
47   th,
48   td {
49     border: 1px solid #000;
50   }
51   h3 {
52     position: relative;
53   }
54 </style>

```

2.3 总结

1. 计算属性与普通函数的区别？

- 答：1、计算属性基于依赖 **有缓存特性**，普通函数 **没有缓存**
- 2、当基于现有的数据计算得到新数据，推荐使用计算属性
- 3、当处理业务逻辑时，推荐使用函数，比如点击事件的处理函数

3. 计算属性的完整写法

3.1 思考

既然计算属性也是属性，能访问，应该也能修改了？

1. 计算属性默认的写法，只能读(展示数据)，不能"改"
2. 如果要"改" → 需要用计算属性的完整写法

```

// 简易写法
const uname = computed(() => {
  // 计算得到新数据
  return '姬霓太美'
})

```

```

// 完整写法 = get + set
const uname = computed({
  get() {
    // 计算得到新数据
    return '姬霓太美'
  },
  set(val) {
    // 一段修改逻辑
    console.log(val)
  }
})

```

3.2 代码演示

```

1  <script setup>
2    import { computed } from 'vue'
3
4    // 完整写法 = get + set
5    const uname = computed({
6      // 使用计算属性时，自动触发get，get内部必须 `return` 计算结果
7      get() {
8        // 计算得到新数据
9        return '姬霓太美'
10     },
11     // 给计算属性赋值时，自动触发set，并接收赋予的新值val
12     set(val) {
13       // 一段修改逻辑
14       console.log(val)
15     }
16   })
17 </script>
18 <template>
19   <input type="text" v-model="uname" />
20 </template>

```

3.3 总结

1. 何时使用计算属性完整写法？

答：当 `修改计算属性` 时，也就是当计算属性配合v-model的时候

2. 完整写法的是？

答： `get + set`

```

1  const xxx = computed({
2    get() {
3      return 计算结果
4    },
5    set(val) {
6
7    }
8  })

```

4. 案例 - 全返反选

4.1 效果

☐ 全选	反选
☐	跑步
☐	看书
☐	撸码
☐	追剧

4.2 静态代码(可复制)

```
1  <script setup>
2    import { ref } from 'vue'
3
4    // 计划列表
5    const planList = ref([
6      { id: 12, name: '跑步', done: false },
7      { id: 76, name: '看书', done: false },
8      { id: 31, name: '撸码', done: false },
9      { id: 49, name: '追剧', done: false }
10   ])
11
12 </script>
13
14 <template>
15   <p>
16     <span>
17       <input type="checkbox" id="all" />
18       <label for="all">全选</label>
19     </span>
20     <button>反选</button>
21   </p>
22   <ul>
23     <li>
24       <input type="checkbox" />
25       <span>xxx</span>
26     </li>
27   </ul>
28 </template>
29
30 <style lang="scss">
31   * {
32     margin: 0;
33     padding: 0;
34   }
35   div {
36     width: 400px;
```

```

37     margin: 100px auto;
38     padding: 15px;
39     font-size: 18px;
40     background: plum;
41     p {
42         display: flex;
43         justify-content: space-between;
44         align-items: center;
45         height: 40px;
46         button {
47             padding: 3px 6px;
48         }
49     }
50 }
51
52 ul {
53     list-style: none;
54     li {
55         display: flex;
56         align-items: center;
57         justify-content: space-between;
58         height: 40px;
59         border-top: 1px solid #ccc;
60         span.completed {
61             color: #ddd;
62             text-decoration: line-through;
63         }
64     }
65 }
66
67 input {
68     margin-right: 8px;
69 }
70 </style>

```

4.3 完整代码

```

1  <script setup>
2    import { computed, ref } from 'vue'
3    // 计划列表
4    const planList = ref([
5      { id: 12, name: '跑步', done: false },
6      { id: 76, name: '看书', done: false },
7      { id: 31, name: '撸码', done: false },
8      { id: 49, name: '追剧', done: false }

```

```

9     ]})
10    // 是否全部选中
11    const isAll = computed({
12      // 使用计算属性自动触发 get
13      get() {
14        // every: 检测每一个
15        return planList.value.every((item) => item.done)
16      },
17      // 修改计算属性自动触发 set
18      set(val) {
19        // val: 给计算属性赋予的新值, 在这里就是表全选复选框的状态
20
21        // 遍历 planList 数组, 把每个小选的 done 属性与 val 保持一致即可
22        planList.value.forEach((item) => (item.done = val))
23      }
24    })
25
26    // 反选
27    const onToggle = () => {
28      // 遍历 planList 数组, 对每个对象的 done 属性取反即可
29      planList.value.forEach((item) => (item.done = !item.done))
30    }
31  </script>
32  <template>
33    <p>
34      <span>
35        <input
36          v-model="isAll"
37          type="checkbox"
38          id="all" />
39        <label for="all">全选</label>
40      </span>
41      <button @click="onToggle">反选</button>
42    </p>
43    <ul>
44      <li
45        v-for="item in planList"
46        :key="item.id">
47        <input
48          type="checkbox"
49          v-model="item.done" />
50        <span :class="{ completed: item.done }">{{ item.name }}</span>
51      </li>
52    </ul>
53  </template>
54
55  <style lang="scss">

```

```
56 // npm i sass -D
57 // npm run dev
58 * {
59     margin: 0;
60     padding: 0;
61 }
62 ul {
63     list-style: none;
64 }
65 #app {
66     width: 400px;
67     margin: 100px auto;
68     padding: 15px 18px;
69     background: plum;
70     p {
71         display: flex;
72         justify-content: space-between;
73         align-items: center;
74         height: 40px;
75         border-bottom: 1px solid #ccc;
76         button {
77             padding: 3px 6px;
78         }
79         input {
80             margin-right: 8px;
81         }
82     }
83     ul {
84         li {
85             display: flex;
86             justify-content: space-between;
87             align-items: center;
88             height: 40px;
89             border-bottom: 1px solid #ccc;
90             span.completed {
91                 color: #ddd;
92                 text-decoration: line-through;
93             }
94         }
95     }
96 }
97 </style>
```

四、侦听器

1. 作用

监视数据变化, 执行DOM操作或异步操作

2. 语法

1. 监视简单类型

```
1  <script setup>
2    import { ref, watch } from 'vue'
3
4    // 搜索框关键字
5    const keyword = ref('')
6
7    // 监视 keyword, 只要 keyword 的值变了, 就会执行回调函数
8    watch(keyword, (newVal, oldVal) => {
9      // newVal: 新值
10     // oldVal: 旧值
11   })
12
13 </script>
14 <template>
15   <div>
16     <input
17       type="text"
18       v-model="keyword"
19       placeholder="请输入关键字" />
20   </div>
21 </template>
```

2. 监视复杂类型

```
1  <script setup>
2    import { reactive, watch } from 'vue'
3
4    // 表单对象
5    const obj = reactive({
6      username: '',
7      password: ''
8    })
9
10   // 监视 obj 的变化
11   watch(obj, (newVal, oldVal) => {
12     console.log(newVal, oldVal)
```

```
13     })
14   </script>
15   <template>
16     <div>
17       <input
18         type="text"
19         v-model="obj.username"
20         placeholder="请输入用户名" />
21     <br/>
22     <br/>
23     <input
24       type="text"
25       v-model="obj.password"
26       placeholder="请输入密码" />
27   </div>
28 </template>
29
```

3. 总结

1. watch的作用是？

答：监视响应式数据的变化, 当数据变了, 针对性的DOM操作或异步操作

五、案例商品管理

1. 效果

序号	名称	价格	数量	小计	操作
1	biangbiang面	15	2	30	删除
暂无数据					
总数量: 10		总金额: 168			

名称:

价格:

数量:

新增

2. 静态代码(可复制)

```
1  <script setup>
2    import { ref } from 'vue'
3
4    // 商品列表
5    const goodsList = ref([
6      { id: 190, name: '魏家凉皮', price: 10, num: 2 },
7      { id: 237, name: '肉夹馍', price: 11, num: 3 },
8      { id: 163, name: '冰峰', price: 3, num: 2 }
9    ])
10 </script>
11
12 <template>
13   <div class="goods-container">
14     <table>
15       <thead>
16         <tr>
17           <th>序号</th>
18           <th>名称</th>
19           <th>价格</th>
20           <th>数量</th>
21           <th>小计</th>
22           <th>操作</th>
23         </tr>
24       </thead>
25       <tbody>
26         <tr>
27           <td>1</td>
28           <td>biangbiang面</td>
29           <td>15</td>
30           <td>2</td>
31           <td class="active">30</td>
32           <td><a href="#">删除</a></td>
33         </tr>
34       </tbody>
35       <tbody>
36         <tr>
37           <td
38             colspan="6"
39             class="empty">
40             暂无数据
41           </td>
42         </tr>
43       </tbody>
44     </table>
```

```
45     <tfoot>
46     <tr>
47         <td colspan="2">总数量: <b>10</b></td>
48         <td colspan="4">总金额: <b>168</b></td>
49     </tr>
50 </tfoot>
51 </table>
52 <form>
53     <div class="form-item">
54         <label class="label">名称: </label>
55         <div class="input">
56             <input
57                 type="text"
58                 placeholder="请输入名称" />
59         </div>
60     </div>
61     <div class="form-item">
62         <label class="label">价格: </label>
63         <div class="input">
64             <input
65                 type="number"
66                 placeholder="请输入价格" />
67         </div>
68     </div>
69     <div class="form-item">
70         <div class="label">数量: </div>
71         <div class="input">
72             <input
73                 type="number"
74                 placeholder="请输入数量" />
75         </div>
76     </div>
77     <div class="form-item">
78         <div class="label"></div>
79         <div class="input">
80             <button class="btn-add">新增</button>
81         </div>
82     </div>
83 </form>
84 </div>
85 </template>
86 <style>
87     .goods-container {
88         display: flex;
89         flex-direction: column;
90         justify-content: center;
91         align-items: center;
```

```
92     width: 700px;
93     margin: 50px auto;
94     font-family: 'Courier New', Courier, monospace;
95 }
96 .goods-container table {
97     flex: 4;
98 }
99 .goods-container table {
100     width: 100%;
101     border-spacing: 0;
102     border-top: 1px solid #ccc;
103     border-left: 1px solid #ccc;
104 }
105 .goods-container table th {
106     background: #f5f5f5;
107 }
108 .goods-container table tr:hover td {
109     background: #f5f5f5;
110 }
111 .goods-container table td,
112 .goods-container table th {
113     padding: 10px;
114     border-bottom: 1px solid #ccc;
115     border-right: 1px solid #ccc;
116     text-align: center;
117 }
118 .goods-container table td a {
119     color: blue;
120     text-decoration: none;
121 }
122 .goods-container table td.active {
123     color: #f00;
124 }
125 .goods-container table tbody .empty {
126     height: 100px;
127     color: #999;
128     font-size: 18px;
129 }
130 .goods-container table tfoot {
131     background: #f5f5f5;
132 }
133 .goods-container form {
134     margin-top: 50px;
135 }
136 .goods-container form .form-item {
137     display: flex;
138     margin-bottom: 20px;
```

```

139     align-items: center;
140 }
141 .goods-container form .form-item .label {
142     width: 60px;
143     font-size: 14px;
144     text-align: right;
145 }
146 .goods-container form .form-item .input {
147     flex: 1;
148 }
149 .goods-container form .form-item input {
150     box-sizing: border-box;
151     width: 250px;
152     height: 36px;
153     border: 1px solid #ccc;
154     padding: 10px;
155     appearance: none;
156     outline: none;
157     color: #999;
158 }
159 .goods-container form .form-item input::placeholder {
160     color: #999;
161 }
162
163 .goods-container form .form-item .btn-add {
164     padding: 6px 12px;
165     border: 1px solid blue;
166     font-size: 14px;
167     border-radius: 4px;
168     background: blue;
169     color: #fff;
170     appearance: none;
171     outline: none;
172     cursor: pointer;
173 }
174 </style>

```

3. 完整代码

```

1  <script setup>
2      import { ref, reactive, watch, computed } from 'vue'
3
4      const KEY = 'bit-goodsSystem__key'
5
6      // 商品列表

```

```

7   const goodsList = ref(
8     JSON.parse(localStorage.getItem(KEY)) || [
9       { id: 190, name: '魏家凉皮', price: 10, num: 2 },
10      { id: 237, name: '肉夹馍', price: 11, num: 3 },
11      { id: 163, name: '冰峰', price: 3, num: 2 }
12    ]
13  )
14
15  // 收集的商品信息
16  const goodsItem = reactive({
17    name: '', // 名称
18    price: '', // 价格
19    num: '' // 数量
20  })
21
22  // 新增
23  const add = () => {
24    // 非空校验
25    if (!goodsItem.name || !goodsItem.price || !goodsItem.num) {
26      return alert('商品名称或价格或数量不能为空')
27    }
28
29    // 给数组添加一个新对象
30    goodsList.value.push({
31      // 展开 goodsItem 对象
32      ...goodsItem,
33      // 新增 id 属性
34      id: Date.now() // 时间戳
35    })
36
37    // 清空输入框
38    goodsItem.name = goodsItem.price = goodsItem.num = ''
39  }
40
41  // 删除
42  const del = (i) => {
43    if (window.confirm('确认删除么?')) {
44      goodsList.value.splice(i, 1)
45    }
46  }
47
48  // 总数量
49  const totalNum = computed(() =>
50    goodsList.value.reduce((prev, item) => prev + item.num, 0)
51  )
52
53  // 总金额

```

```

54     const totalAmount = computed(() => {
55         return goodsList.value.reduce((prev, item) => {
56             return prev + item.price * item.num
57         }, 0)
58     })
59
60     watch(
61         goodsList,
62         (newVal) => {
63             localStorage.setItem(KEY, JSON.stringify(newVal))
64         },
65         {
66             deep: true
67         }
68     )
69 </script>
70
71 <template>
72     <div class="goods-container">
73         <table>
74             <thead>
75                 <tr>
76                     <th>序号</th>
77                     <th>名称</th>
78                     <th>价格</th>
79                     <th>数量</th>
80                     <th>小计</th>
81                     <th>操作</th>
82                 </tr>
83             </thead>
84             <tbody v-if="goodsList.length > 0">
85                 <tr
86                     v-for="(item, index) in goodsList"
87                     :key="item.id">
88                     <td>{{ index + 1 }}</td>
89                     <td>{{ item.name }}</td>
90                     <td>{{ item.price }}</td>
91                     <td>{{ item.num }}</td>
92                     <td :class="{ active: item.price * item.num >= 30 }">
93                         {{ item.price * item.num }}
94                     </td>
95                     <td>
96                         <a
97                             href="#"
98                             @click="del(index)"
99                             >删除</a>
100                     </td>

```



```
101         </td>
102     </tr>
103 </tbody>
104 <tbody v-else>
105     <tr>
106         <td
107             colspan="6"
108             class="empty">
109             暂无数据
110         </td>
111     </tr>
112 </tbody>
113
114 <tfoot>
115     <tr>
116         <td colspan="2">
117             总数量: <b>{{ totalNum }}</b>
118         </td>
119         <td colspan="4">
120             总金额: <b>{{ totalAmount }}</b>
121             &yen;
122         </td>
123     </tr>
124 </tfoot>
125 </table>
126 <form>
127     <div class="form-item">
128         <label class="label">名称: </label>
129         <div class="input">
130             <input
131                 type="text"
132                 placeholder="请输入名称"
133                 v-model.trim="goodsItem.name" />
134         </div>
135     </div>
136     <div class="form-item">
137         <label class="label">价格: </label>
138         <div class="input">
139             <input
140                 type="number"
141                 placeholder="请输入价格"
142                 v-model.number="goodsItem.price" />
143         </div>
144     </div>
145     <div class="form-item">
146         <div class="label">数量: </div>
147         <div class="input">
```

```

148         <input
149             type="number"
150             placeholder="请输入数量"
151             v-model.number="goodsItem.num" />
152     </div>
153 </div>
154 <div class="form-item">
155     <div class="label"></div>
156     <div class="input">
157         <button
158             class="btn-add"
159             @click.prevent="add">
160             新增
161         </button>
162     </div>
163 </div>
164 </form>
165 </div>
166 </template>

```

六、案例轮播图

1. 效果



2. 静态代码(可复制)

代码块

```

1  <script setup>
2    const images = [
3      'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0f9be4b071da45d9a0c2.jpg',
4      'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0fa3e4b071da45d9a0c3.jpg',

```

```
5     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa10ace4b071da45d9a0c4.jpg'
6 ]
7 </script>
8
9 <template>
10     <div class="banner">
11         <ul>
12             <li class="active">
13                 
15                 </li>
16             </ul>
17             <div class="indicator">
18                 <span class="active"> </span>
19             </div>
20             <div class="ctrl">
21                 <a
22                     href="javascript:;"
23                     class="btn prev"
24                     >&lt;</a>
25                 ><a
26                     href="javascript:;"
27                     class="btn next"
28                     >&gt;</a>
29             >
30         </div>
31     </div>
32 </template>
33
34 <style>
35     * {
36         margin: 0;
37         padding: 0;
38     }
39     a {
40         text-decoration: none;
41         color: #000;
42     }
43
44     .banner {
45         position: relative;
46         width: 1200px;
47         height: 337px;
48         margin: 150px auto;
49     }
```

```

50 .banner ul {
51   width: 100%;
52   height: 100%;
53   list-style: none;
54 }
55
56 .banner ul li {
57   position: absolute;
58   top: 0;
59   left: 0;
60   width: 100%;
61   height: 100%;
62   opacity: 0;
63   transition: opacity 1.2s;
64 }
65 .banner ul li.active {
66   opacity: 1;
67 }
68
69 .banner .indicator {
70   display: flex;
71   justify-content: center;
72   position: absolute;
73   left: 0;
74   bottom: 20px;
75   width: 100%;
76 }
77
78 .banner .indicator span {
79   width: 30px;
80   height: 2px;
81   margin: 0 5px;
82   cursor: pointer;
83   background: rgba(255, 255, 255, 0.6);
84 }

```

3. 完整代码

代码块

```

1 <script setup>
2   import { ref } from 'vue'
3   const i = ref(0)
4   const images = [
5     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0f9be4b071da45d9a0c2.jpg',

```

```
6     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa0fa3e4b071da45d9a0c3.jpg',
7     'https://bit-1256306791.cos.ap-
chengdu.myqcloud.com/fffa10ace4b071da45d9a0c4.jpg'
8   ]
9
10  const prev = () => {
11    i.value--
12    if (i.value <= -1) {
13      i.value = images.length - 1
14    }
15  }
16
17  const next = () => {
18    i.value++
19    if (i.value >= images.length) {
20      i.value = 0
21    }
22  }
23
24  let timer = null
25
26  const play = () => {
27    timer = setInterval(() => {
28      next()
29    }, 3000)
30  }
31
32  play()
33
34  const stop = () => {
35    clearInterval(timer)
36  }
37 </script>
38
39 <template>
40   <div
41     class="banner"
42     @mouseenter="stop"
43     @mouseleave="play">
44     <ul>
45       <li
46         v-for="(url, index) in images"
47         :class="{ active: index === i }">
48         
49       </li>
50     </ul>
```

```
51     <div class="indicator">
52         <span
53             v-for="(n, index) in images.length"
54             :key="n"
55             :class="{ active: index === i }"
56             @click="i = index">
57             </span>
58     </div>
59     <div class="ctrl">
60         <a
61             href="javascript:;"
62             class="btn prev"
63             @click="prev"
64             >&lt;</a>
65         <a
66             href="javascript:;"
67             class="btn next"
68             @click="next"
69             >&gt;</a>
70     >
71     </div>
72 </div>
73 </template>
74
75 <style>
76     * {
77         margin: 0;
78         padding: 0;
79     }
80     a {
81         text-decoration: none;
82         color: #000;
83     }
84
```